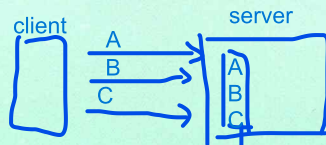


개발 환경 설정하고 GPT API 시작하기

이번 장에서는 처음으로 GPT API를 활용해 인공지능의 답변을 생성해 봅니다. 독자 여러분이 학습하는 데 도움되도록 최대한 동일한 환경에서 프로젝트를 진행할 수 있도록 소개하겠습니다.

- 개발
- 02-1 파이썬 프로그래밍 환경 설정하기
- 02-2 GPT API 시작하기



요청 방법을 서버에서 정해 놓음.

02-1 파이썬 프로그래밍 환경 설정하기

대규모 언어 모델 기반의 애플리케이션 개발에는 파이썬이 가장 널리 사용되고 있습니다. GPT를 만든 오픈AI는 파이썬 기반 라이브러리를 제공하며 언어 모델을 쉽게 사용할 수 있도록 돕는 올라마, 랭체인, 랭그래프와 같은 라이브러리나 프레임워크도 모두 파이썬에서 활용할 수 있도록 만들어졌습니다. 이 기술에 대한 문서들 역시 대부분 파이썬을 중심으로 작성되어 있죠. 따라서 이 책에서도 파이썬을 기반으로 실습합니다. [python + Venv](#)

실습하기 전에 파이썬과 비주얼 스튜디오 코드를 설치하고 프로젝트에서 사용할 가상 환경을 설정해 앞으로 사용할 라이브러리 버전을 프로젝트별로 관리할 수 있도록 해 보겠습니다.

Do it! 실습 파이썬 설치하기

먼저 파이썬을 설치하겠습니다. 이 책은 파이썬 3.12 버전 환경에서 진행했습니다. 만약 다른 다른 파이썬 버전이 설치되어 있다면 3.12 버전으로 [이 책에서 사용하는 라이브러리 가운데 일부는 파이썬 3.13 버전에서 아직 호환되지 않습니다.](#) 설정하길 바랍니다.

1. 파이썬 공식 웹 사이트(<https://www.python.org/>)에 접속하고 [Downloads] 버튼을 클릭합니다.



2. Release version에서 [Python 3.12.9] 버전을 찾아 [Download]를 클릭합니다.

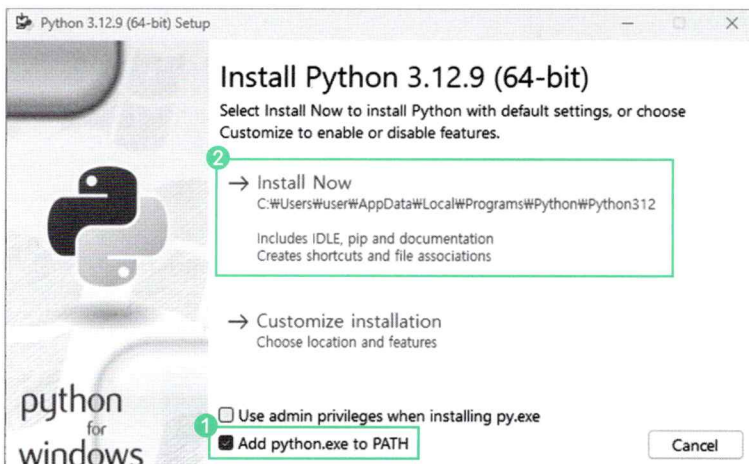
Python releases by version number:		
Release version	Release date	
Python 3.13.2	Feb. 4, 2025	Download
Python 3.12.9	Feb. 4, 2025	Download
Python 3.12.8	Dec. 3, 2024	Download

3. Files에서 컴퓨터 환경에 맞는 설치 파일을 클릭해 내려받습니다. 자신의 PC가 윈도우 환경이라면 [Windows Installer (64bit)]를, 맥 환경이라면 [macOS 64-bit universal2 installer]를 선택해서 내려받으세요.

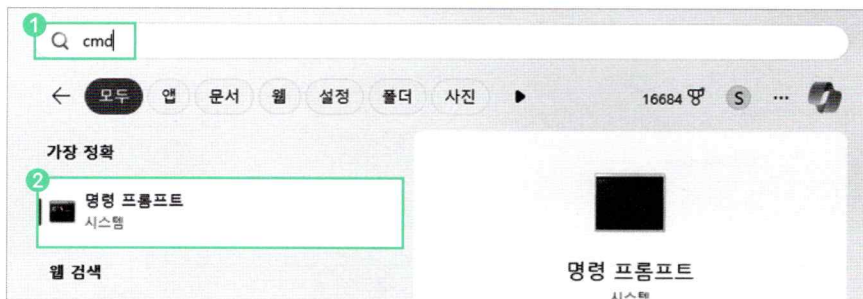
◆ 이 책은 윈도우 환경에서 실습합니다.

Files			
Version	Operating System	Description	MD5 Sum
Gzipped source tarball	Source release		ce613c72fa9b32fb4f109762d61b249b
XZ compressed source tarball	Source release		880942124f7d5c01e7b65cbad62dc873
macOS 64-bit universal2 installer	macOS	for macOS 10.9 and later	fe3db9abb5c61010a9050d83bfc8372
Windows installer (64-bit)	Windows	Recommended	1cfb1bbf96007b12b98db895dcd86487

4. 설치 파일을 실행하고 [Add python.exe to PATH] 앞을 클릭해 체크 표시한 뒤 [Install Now]를 클릭합니다.



5. 파이썬 설치를 완료한 후 윈도우 검색 창에 'cmd'를 입력하고 [명령 프롬프트]를 클릭해 실행합니다.



6. 명령 프롬프트에서 'python'을 입력하면 파이썬의 3.12 버전이 잘 설치된 것을 확인할 수 있습니다.

```
명령 프롬프트

Microsoft Windows [Version 10.0.22631.4602]
(c) Microsoft Corporation. All rights reserved.

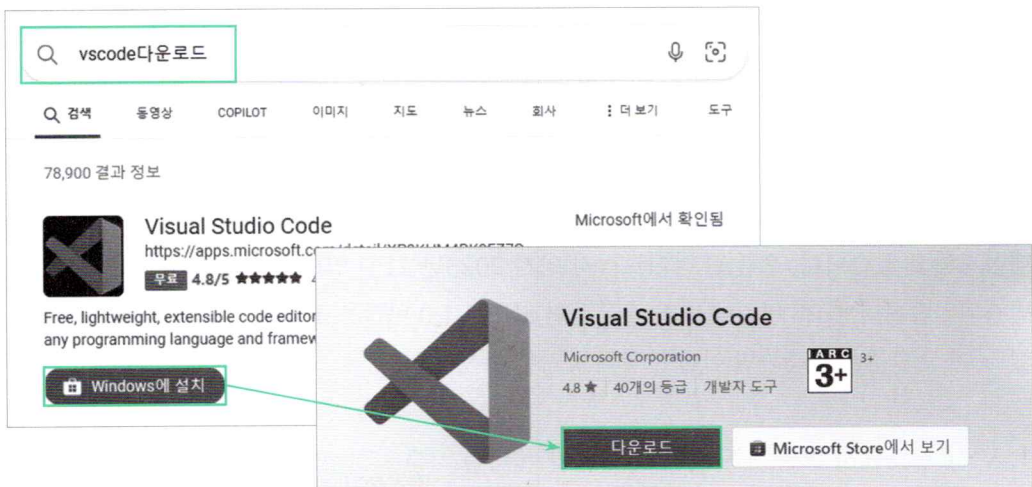
C:/Users/사용자명>python
Python 3.12.9 (tags/v3.12.9:fdb8142, Feb  4 2025, 15:27:58) [MSC v.1942 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Do it! 실습 비주얼 스튜디오 코드 설치하기

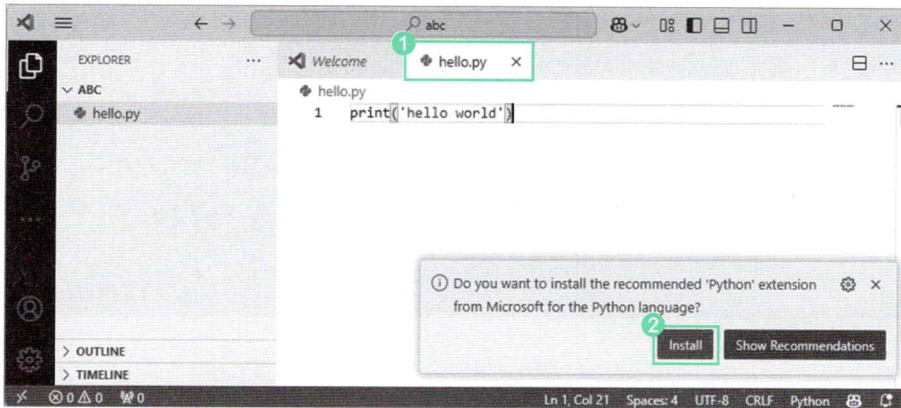
이 책에서는 통합 개발 환경 **Integrated Development Environment, IDE**으로 비주얼 스튜디오 코드(이하 VS Code)를 사용합니다. 물론 VS Code가 아닌 익숙한 IDE를 사용해도 괜찮습니다.

1. 엣지 브라우저에서 'vscode 다운로드'를 검색하면 설치 버튼이 나옵니다. [Windows에 설치] 버튼을 클릭하고 마이크로소프트 스토어에서 [다운로드]를 선택합니다. VS Code는 무료로 설치할 수 있습니다.

◆ 비주얼 스튜디오 코드 웹 사이트(<https://code.visualstudio.com/>)에서도 설치 파일을 내려받을 수 있습니다.



2. 설치가 완료되면 VS Code를 실행하고 확장자명이 .py인 파이썬 파일을 생성해 봅시다. 그러면 VS Code의 파이썬 익스텐션을 설치하라는 창이 표시됩니다. [Install]을 클릭해 설치합니다.



◆ 이 책의 VS Code 테마는 가독성을 위해 밝은 버전으로 설정했습니다. 만약 테마를 변경하고 싶다면 [설정] → 테마 → 색 테마]에서 선택하세요.

이제 AI 에이전트 개발을 위해 파이썬 프로그래밍을 할 수 있는 준비를 갖추었습니다.

Do it! 실습 가상 환경 만들기

프로젝트에서 사용할 가상 환경을 만들어 봅시다. 파이썬 개발을 위한 가상 환경을 만들면 다른 프로젝트에 영향을 주지 않고 작업할 수 있습니다. 예를 들어 A라는 프로젝트에는 오픈AI 라이브러리의 0.28 버전을 사용하고, B라는 프로젝트에서는 1.0 버전을 사용하는 상황을 가정해 봅시다. 만약 가상 환경을 만들지 않으면 오픈AI 라이브러리를 1.0 버전으로 설치할 때 A 프로젝트의 기존 설정이 영향을 받을 수 있습니다. 라이브러리 버전에 따라 동작이 달라지거나 사용하는 방법이 변경될 수 있으므로 A 프로젝트에서 잘 동작하던 코드가 더 이상 실행되지 않는 문제가 생길 수 있습니다. 생성형 AI 관련 라이브러리와 패키지는 몇 주 단위로 업데이트되고 있습니다. 오픈AI 라이브러리 역시 2023년 11월 업데이트되면서 사용법이 크게 변경되었습니다. 0.28 버전에서 잘 작동하던 코드도 1.0 버전을 설치한 이후에는 오류가 발생했죠. 가상 환경을 구축하면 이런 문제를 방지할 수 있습니다.

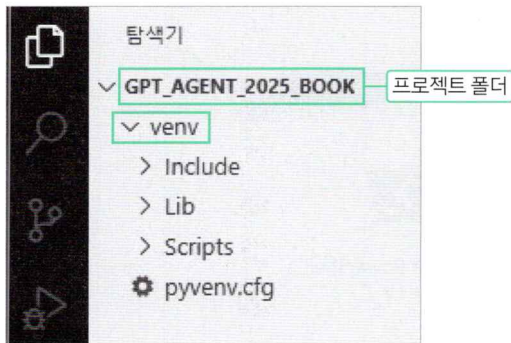
1. VS Code에서 프로젝트 폴더로 사용할 폴더를 엽니다. 그리고 **[Ctrl] + `**을 눌러 **터미널 창**을 열고 다음과 같이 가상 환경 이름을 입력하세요.

```
> python -m venv venv 가상 환경명
```

venv만들면
상위 디렉토리 아래에서만 작동

실행하면 다음과 같이 venv라는 폴더가 생깁니다. 이 폴더 안에 가상 환경을 위한 파일이 준비됩니다.

◆ 프로젝트 폴더는 이름은 원하는 것으로 설정해도 됩니다.



2. VS Code의 터미널 창에 다음과 같이 입력해 가상 환경을 활성화합니다.

- ◆ 가상 환경을 만들 때 venv가 아니라 다른 이름을 사용했다면 venv 대신 그 이름을 입력합니다.
- ◆ 맥 환경에서는 source venv/bin/activate 라고 입력하세요.

```
> .\venv\Scripts\activate
```

경로 앞에 (venv)가 표시되면 성공입니다.

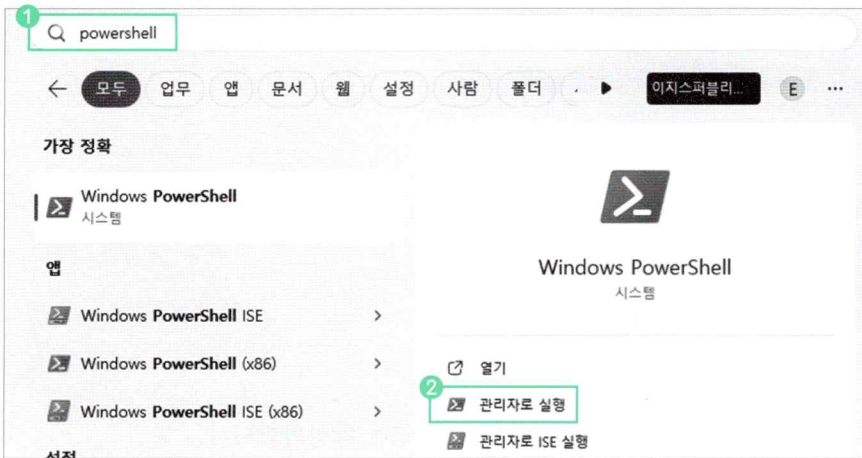
```
문제  출력  디버그 콘솔  터미널  포트
● PS C:\github\gpt_agent_2025_book> .\venv\Scripts\activate
○ (venv) PS C:\github\gpt_agent_2025_book> []
```

가상 환경을 만들 때 오류가 발생한다면?

윈도우에서 가상 환경을 처음 만들면 다음과 같은 오류 메시지가 보일 수 있습니다.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS
powershell + v [ ] [ ] ... ^ x
PS C:\github\gpt_agent_2025_book> python -m venv venv
PS C:\github\gpt_agent_2025_book> .\venv\Scripts\activate
.\venv\Scripts\activate : 이 시스템에서 스크립트를 실행할 수 없으므로 C:\github\gpt_agent_2025_book\venv\Scripts\Activate.ps1
파일을 로드할 수 없습니다. 자세한 내용은 about_Execution_Policies(https://go.microsoft.com/fwlink/?LinkID=135170)를 참조하십시오.
위치 줄:1 문자:1
+ .\venv\Scripts\activate
+ ~~~~~
+ CategoryInfo          : 보안 오류: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\github\gpt_agent_2025_book> []
```

1. 오류를 해결해 보겠습니다. 먼저 윈도우 검색 창에 'powershell'을 검색하고 [관리자로 실행]을 클릭합니다.



2. 윈도우 파워셸 powershell이 열리면 `get-ExecutionPolicy`를 입력해 현재 실행 정책을 확인하세요.

```
윈도우 파워셸
PS C:\WINDOWS\system32> get-ExecutionPolicy
```

3. Restricted는 스크립트 실행이 허용되지 않는 상태를 의미합니다. 실행 정책을 변경하기 위해 `Set-ExecutionPolicy RemoteSigned`를 입력하고 이어서 `y`를 입력합니다.

```
윈도우 파워셸
PS C:\WINDOWS\system32> get-ExecutionPolicy
Restricted
PS C:\WINDOWS\system32> Set-ExecutionPolicy RemoteSigned
실행 규칙 변경
실행 정책은 신뢰하지 않는 스크립트로부터 사용자를 보호합니다. 실행 정책을 변경하면 about_Execution_Policies 도움말
항목(https://go.microsoft.com/fwlink/?LinkID=135170)에 설명된 보안 위험에 노출될 수 있습니다. 실행 정책을
변경하시겠습니까?
[Y] 예(Y) [A] 모두 예(A) [N] 아니요(N) [L] 모두 아니요(L) [S] 일시 중단(S) [?] 도움말
(기본값은 "N"):
```

4. 이제 VS Code 터미널 창에 `.\venv\Scripts\activate`를 입력하면 오류 없이 가상 환경이 설정됩니다.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS powershell + v [ ] [ ] ... ^ x
PS C:\github\gpt_agent_2025_book> python -m venv venv
PS C:\github\gpt_agent_2025_book> .\venv\Scripts\activate
.\venv\Scripts\activate : 이 시스템에서 스크립트를 실행할 수 없으므로 C:\github\gpt_agent_2025_book\venv\Scripts\Activate.ps1
파일을 로드할 수 없습니다. 자세한 내용은 about_Execution_Policies(https://go.microsoft.com/fwlink/?LinkID=135170)를 참조하십시
오.
위치 줄:1 문자:1
+ .\venv\Scripts\activate
+ ~~~~~
+ CategoryInfo          : 보안 오류: (:) [], PSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\github\gpt_agent_2025_book> .\venv\Scripts\activate
(venv) PS C:\github\gpt_agent_2025_book> [ ]
```

이제 앞으로 모든 실습은 이 가상 환경에서 진행됩니다. 따로 설명하지 않더라도 가상 환경을 활성화한 상태에서 작업하세요.

02-2 GPT API 시작하기

이 책에서는 오픈AI의 API를 활용해 GPT에 기반한 프로그램을 만듭니다. 시작하기에 앞서 API와 오픈AI의 API를 자세히 알아본 후 오픈 AI의 API를 활용해 보겠습니다.

API란?

API(Application Programming Interface)는 프로그램 간의 대화에 사용하기 위해 미리 정해 놓은 규칙 체계입니다. 예를 들어 날씨 앱을 생각해 봅시다. 스마트폰에 설치된 날씨 앱에는 사실 날씨를 예측하는 프로그램이나 날씨 정보 데이터가 없습니다. 그 대신 기상청에서 제공하는 서버에 미리 약속한 규칙으로 데이터를 요청하면 해당 지역의 날씨 정보를 받아 올 수 있습니다. 이처럼 정보를 가지고 있는 누군가 API 서비스를 제공하면 다른 사람이나 프로그램이 관련 정보를 가져올 수 있습니다.

API를 사용하면 이미 검증된 인증, 결제, 데이터 처리 등의 기능을 안정적으로 재사용할 수 있어 중복해서 개발하지 않고 빠르게 진행할 수 있습니다. 또한 개인 PC나 스마트폰에서 실행하기 어려운 프로그램도 대용량 컴퓨터 자원을 갖춘 서버에서 처리한 후 결과만 받아 오는 방식으로 개발할 수 있습니다. API 서비스 제공자는 인증 키를 사용해서 원치 않는 접근을 차단하거나 유료 서비스 형태로 운영할 수 있습니다.

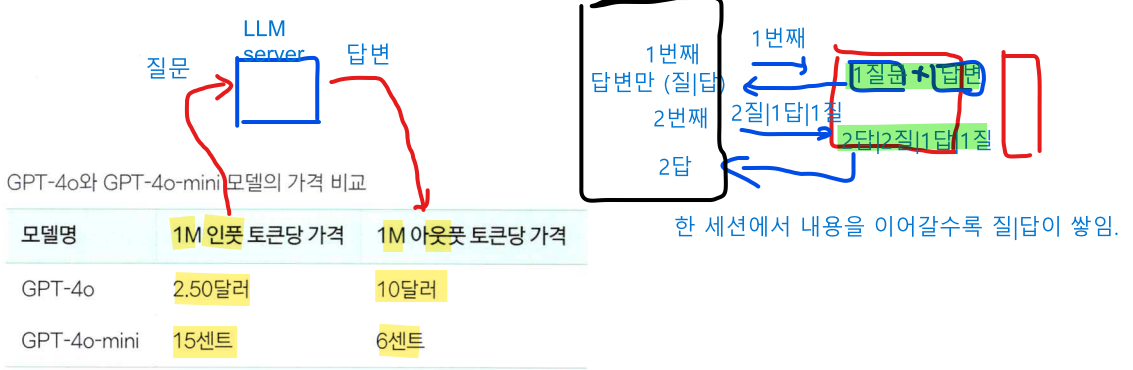
오픈AI의 API

구독으로는 애플리케이션 개발이 안 됨..(클라우드 코드는 개인용 가능)

오픈AI에서 제공하는 대부분의 모델은 API 방식이 아니면 실행할 수 없을 정도로 대규모 모델입니다. 오픈AI는 자신이 보유한 막대한 컴퓨터 자원으로 언어 모델인 GPT, 이미지 생성 모델인 달리 DALL-E, 동영상 생성 모델인 소라 Sora 등의 대규모 모델을 실행하고 사용자가 결과를 받아 갈 수 있는 API 서비스를 제공합니다.

오픈AI의 API를 이용하면 최신 모델을 누구나 쉽게 사용할 수 있습니다. 이 책의 실습에서는 주로 GPT-4o와 GPT-4o-mini 모델을 이용합니다. 두 모델의 가격은 다음과 같습니다.

◆ 추후 가격이 변동되거나 최신 모델이 출시될 수도 있으니 더 자세한 정보는 공식 문서를 참고하세요
(<https://openai.com/api/pricing/>).



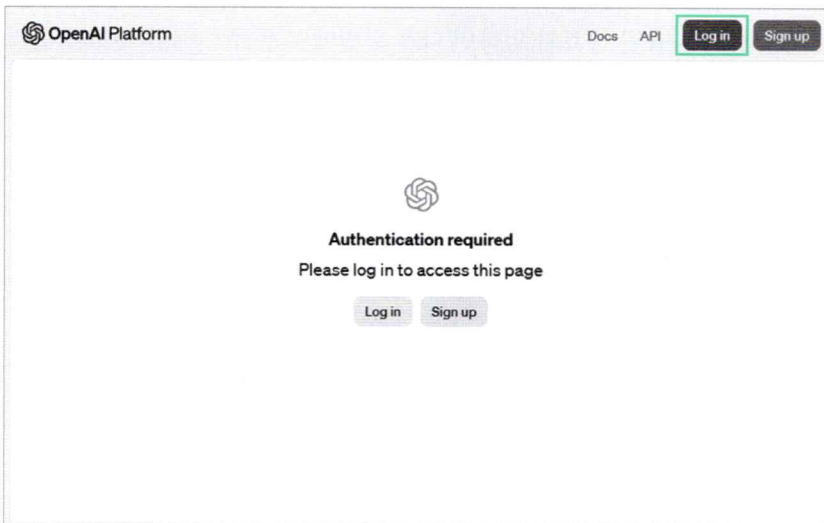
오픈AI에서는 토큰을 단위로 금액을 산정하여 사용자에게 청구합니다. 언어 모델에서 토큰 **token**은 텍스트의 일정 단위를 의미합니다. 예를 들어 영어에서는 단어, 문장 부호, 공백 등을 기준으로 할 수 있고 한글은 음절이나 의미 단위로 쪼갤 수 있습니다. 긴 글을 입력하거나 출력할 경우 더 많은 토큰을 사용하는데 이에 따라 비용이 증가한다고 생각하면 됩니다.

✦ 문장을 토큰으로 나누는 방식은 여러 가지 있는데 구체적인 방법은 이 책의 범위를 넘어선 내용이므로 여기서는 다루지 않습니다.

Do it! 실습 오픈AI의 API 키 발급받기

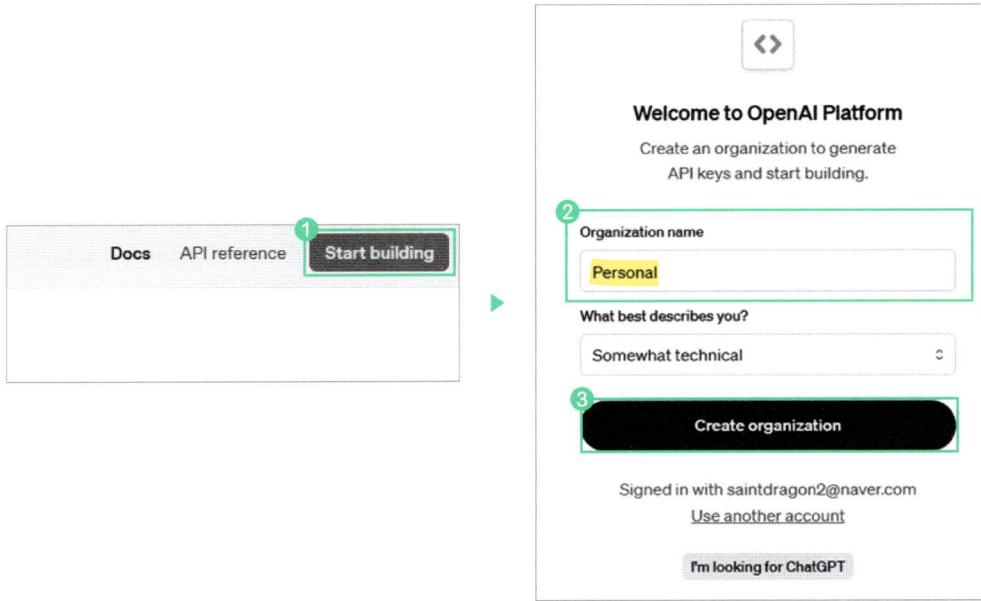
오픈AI의 API를 사용하려면 API 키를 발급받아야 합니다.

1. 오픈AI 플랫폼 웹 사이트(<https://platform.openai.com/api-keys>)에 접속합니다. 다음 화면이 나타나면 오른쪽 위에 있는 [Log in] 버튼을 클릭해 로그인합니다.

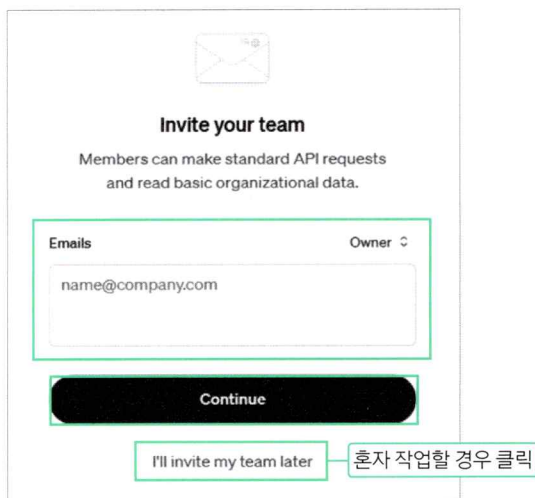


2. 로그인했다면 오른쪽 상단의 [Start building]을 클릭합니다. 그리고 개인 작업인지 회사 소속 작업인지 결정해 [Organization name] 아래 빈칸에 입력하고 [Create organization] 버튼을 클릭합니다.

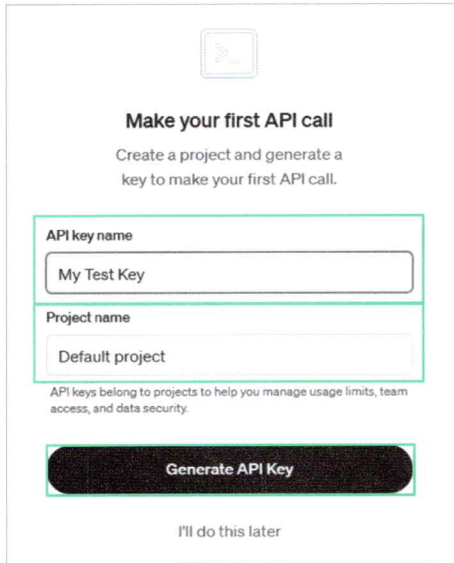
◆ 웹 사이트가 업데이트되면 화면 구성이 달라질 수 있습니다.



만약 오픈AI의 API 키를 함께 사용할 동료가 있다면 다음 화면에서 동료 이메일 주소를 입력하고 [Continue] 버튼을 클릭해 초대하세요. 혼자 작업한다면 [I'll invite my team later]를 선택하고 넘어가세요.



3. API 키 이름과 앞으로 만들 프로젝트명을 입력하고 [Generate API Key] 버튼을 클릭합니다. API 키 이름과 프로젝트명은 원하는 값을 입력하면 됩니다. 여러 API 키를 각기 다른 프로젝트에 사용할 수 있으므로 키 이름으로 구분할 수 있도록 만듭니다.



Make your first API call
Create a project and generate a key to make your first API call.

API key name
My Test Key

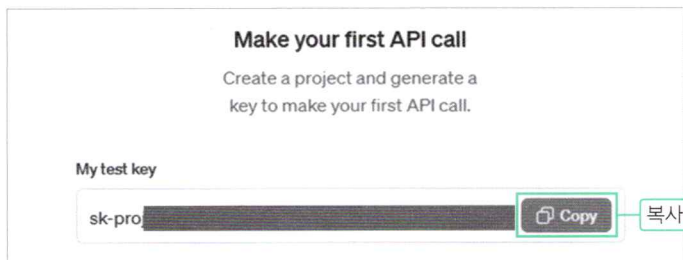
Project name
Default project

API keys belong to projects to help you manage usage limits, team access, and data security.

Generate API Key

[I'll do this later](#)

4. 다음과 같이 API 키가 발급됩니다. 생성한 키는 팝업 창을 닫으면 다시 확인 할 수 없으므로 [Copy] 버튼을 클릭해서 복사해 두세요.



Make your first API call
Create a project and generate a key to make your first API call.

My test key

sk-proj [redacted]

Copy 복사

6. 결제에 사용할 카드 정보를 입력하고 [Add payment method]를 클릭하면 API 크레딧 구매가 완료됩니다.

Add a payment method to buy credits

Add a payment method

Add your credit card details below. This card will be saved to your account and can be removed at any time.

카드 번호
1234 1234 1234 1234

만료 날짜
MM / YY

보안 코드
CVC

성명
이성용

국가 또는 지역
미국

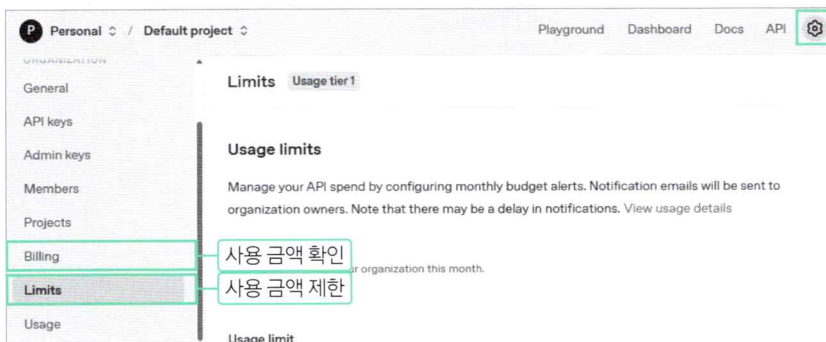
주소

☐ I'm purchasing as a business

Cancel Add payment method

★ 한 걸음 더! 사용 요금을 제한할 수 있나요?

요금에 걱정된다면 오픈AI 플랫폼 웹 사이트(<https://platform.openai.com/api-keys>)에서 설정 아이콘을 클릭하고 ORGANIZATION 영역의 [Limits]를 선택합니다. Usage limit에서 사용 요금을 제한할 수 있습니다. 설정한 금액에 도달하면 사용자에게 메일이 전송됩니다. 현재까지 사용량은 [Billing]에서 확인할 수 있습니다. ◆ 이 책의 실습을 따라 하는 데는 몇천 원이면 충분합니다.



Do it! 실습 오픈AI의 API 키로 질문하고 답변받기

■ 결과 파일: chap02/sec02/gpt_basic.py

생성한 API 키를 사용해 언어 모델에게 질문하고 답변을 받아 보겠습니다.

1. VS Code 터미널 창에 다음처럼 입력해 오픈AI 라이브러리를 설치합니다.

◆ 이 책에서는 1.58.1 버전의 openai 라이브러리를 사용합니다. 최신 버전을 사용하고 싶다면 `pip install openai`라고 입력하세요.

```
(venv) > pip install openai==1.58.1
```

2. chap02 폴더를 만들고 gpt_basic.py 파일을 생성한 후 코드를 입력합니다. 다음은 언어 모델이 사용자가 입력한 텍스트에 대해 형식으로 응답을 생성하는 기능인 Chat Completion을 사용해서 GPT에게 2022년 월드컵 우승 팀을 물어보고 답변을 받아 오는 예제입니다. 이때 `api_key`에는 앞의 실습에서 복사해 둔 자신의 API 키를 넣으면 됩니다.

GPT에게 월드컵 우승팀 물어보기

gpt_basic.py

```
from openai import OpenAI

api_key = 'sk-proj-*****'

client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    temperature=0.1,
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "2022년 월드컵 우승 팀은 어디야?"},
    ]
)

print(response)

print('----')
print(response.choices[0].message.content) # response의 내용만 출력
```

- ❶ OpenAI()로 클라이언트를 생성할 때 입력한 `api_key`를 사용합니다.
- ❷ `client.chat.completions.create()`에 포함된 값 중에서 `model`은 어떤 언어 모델을 사용할지 정하는 부분입니다. 여기서는 `gpt-4o`로 설정했습니다.
- ❸ 매개변수 중에서 `temperature`는 문장을 생성할 때 무작위성을 조절합니다. 0에 가까울수록 안정적이고 일관된 답변을 생성하며, 1에 가까울수록 창의적이고 일관되지 않은 답변이 나옵니다. 정확한 답변을 원할 때는 0에 가깝게, 참신한 답변을 원할 때는 1에 가깝게 소숫점 단위로 설정하면 됩니다.
- ❹ `messages`는 GPT가 과거의 대화를 기반으로 적절한 응답을 생성하는 데 필요한 매개변수입니다. GPT가 맥락에 맞게 응답을 생성하도록 역할(`role`)과 내용(`content`)을 딕셔너리 형태로 만들어 리스트로 쌓아 API로 보냅니다. 대화의 각 부분은 `role`로 구분하며 기본적으로 `system`, `user`, `assistant`의 3가지 역할이 있습니다. `system`은 GPT의 역할을 설정합니다. `user`은 언어 모델과 대화를 나누는 사용자를 뜻하며 `assistant`는 언어 모델이 제공하는 답변을 의미합니다. 여기서는 도우미 역할을 부여하기 위한 기본적인 `content`를 작성합니다. 이 `content`는 사용자가 물어본 질문을 담고 있습니다.
- ❺ '----'은 출력 결과를 구분하는 선입니다. 실행 결과에서 '----'의 위쪽은 `response`의 전체 내용을 출력하고 아래쪽은 `response.choices[0].message.content`를 이용해 답변 내용만 출력합니다. 반환된 `response`의 전체 내용에는 사용한 토큰 수나 언어 모델의 역할 등 여러 정보가 있습니다.

이 코드를 실행하면 다음과 같이 답변이 출력됩니다. `assistant`는 GPT가 제공하는 답변을 의미합니다. '2022년 월드컵 우승팀은 어디야?'라고 질문하면 다음처럼 GPT의 답변과 함께 `role`이 `assistant`로 설정되어 출력됩니다.

◆ 출력되는 답변은 조금씩 다를 수 있습니다.

```
ChatCompletion(id='chatcmpl-9g5M1FzkC3lAbpBpPwfMmHgeoEML0', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='2022년 월드컵 축구 대회는 카타르에서 개최되었습니다. 이번 대회의 우승팀은 아르헨티나입니다. 아르헨티나는 결승전에서 프랑스를 상대로 승리하여 우승을 차지했습니다.', role='assistant', function_call=None, tool_calls=None))], created=1719817385, model='gpt-4o-2024-05-13', object='chat.completion', service_tier=None, system_fingerprint='fp_d576307f90', usage=CompletionUsage(completion_tokens=58, prompt_tokens=30, total_tokens=88))

----

2022년 월드컵 축구 대회는 카타르에서 개최되었습니다. 이번 대회의 우승팀은 아르헨티나입니다. 아르헨티나는 결승전에서 프랑스를 상대로 승리하여 우승을 차지했습니다.
```

★ 한 걸음 더!

GPT의 여러 모델

GPT-3.5-turbo가 처음 나왔을 때 많은 사람이 성능에 감탄했고 이후 몇 개월 사이에 GPT-4-turbo와 GPT 4o가 잇따라 출시되면서 엄청난 발전을 보여 줬습니다. 이전에 다빈치^{davinci}나 GPT-2 등의 모델도 있었지만 현재는 특별한 경우가 아니면 굳이 사용하지 않습니다. 최근에는 GPT-4o를 경량화한 GPT-4o mini가 나오면서 GPT-3.5 계열의 모델도 필요성이 없어졌습니다.

저는 오픈AI의 모델 가운데 GPT-4o 또는 GPT-4o mini를 추천합니다. GPT-4o는 GPT-4-turbo보다 가격이 저렴하면서 성능이 뛰어납니다. 최신 데이터를 학습했다는 장점도 있죠. 만약 토큰 비용이 부담된다면 GPT-4o mini를 고려해 볼 수 있습니다. GPT-4o mini는 GPT-4o에 비해 30배 정도 저렴합니다.

GPT-4o와 GPT-4o-mini 모델 비교

구분	GPT-4o	GPT-4o mini
인풋 컨텍스트 윈도우	128K 토큰	128K 토큰
최대 아웃풋 토큰	2,048 토큰	16.4 토큰
출시일	2024년 5월 13일	2024년 7월 18일
1M 당 인풋 토큰당 가격	2.50달러	15센트
1M 당 아웃풋 토큰당 가격	10달러	6센트

이 책의 실습은 GPT-4o를 기준으로 하며, GPT-4o mini나 GPT-3.5-turbo를 사용할 때에는 따로 설명하겠습니다.

Do it! 실습 API 키 관리하기

■ 결과 파일: sec02/loadenv_gpt_basic.py

오픈AI의 API 키는 반드시 다른 사람에게 노출되지 않도록 관리해야 합니다. 이 키가 유출되면 다른 사람이 사용할 수 있으며 그 비용은 나에게 청구될 수 있습니다. 특히 깃허브 같은 서비스에 코드를 공개할 때 API 키까지 함께 업로드된다면 큰일이겠죠.

막자 -> .env 파일만 key 저장(.gitignore에 env 경로 저장.)

보안상의 이유뿐만 아니라 코드에 API 키를 그대로 적는 것은 관리 면에서도 좋지 않습니다. 만약 여러 파일에 API 키를 사용하는 코드가 나뉘어 있다면 오픈AI의 API 키를 변경할 때마다 해당하는 파일을 찾아서 일일이 수정해야 합니다. 이런 문제를 해결하기 위해 파이썬 개발자는 대부분 `python-dotenv` 라이브러리를 이용합니다. 이 라이브러리는 `.env` 파일에 있는 키-값 쌍을 읽어서 환경 변수로 설정해 줍니다.

1. 라이브러리를 사용하기 위해 먼저 VS Code 터미널 창에서 `python-dotenv`를 설치합니다.

```
> pip install python-dotenv
```

2. 프로젝트 폴더에 `.env` 파일을 만들고 다음처럼 작성합니다. 다음 코드에서 `YOUR-OPENAI-API-KEY`라고 표시된 부분에 자신의 API 키를 넣으세요. `.env` 파일은 파이썬 문법으로 작성되지 않으므로 API 키는 문자열을 의미하는 따옴표 없이 붙여 넣으면 됩니다.

```
OPENAI_API_KEY=YOUR-OPENAI-API-KEY API 키 넣기
```

3. `.env` 파일을 파이썬 코드에서 불러와 사용합니다. 앞에서 만들었던 `gpt_basic.py` 코드에서 `client` 선언 부분을 수정합니다. 이때 `os.getenv`에 `OPENAI_API_KEY`라고 입력한 키와 `.env`의 키가 동일해야 합니다.

API 키 따로 관리하기

gpt_basic.py

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv('OPENAI_API_KEY')
client = OpenAI(api_key=api_key)
(... 생략 ...)
```

4. 깃을 사용하고 있다면 프로젝트 폴더의 .gitignore 파일에 .env 파일이 지정되어 있는지 확인하세요. 만약 .gitignore 파일이 없다면 프로젝트 폴더에 이 파일을 생성하고 다음과 같이 추가합니다. 이렇게 하면 깃에서 .env 파일을 예외로 처리하므로 .env 파일이 실수로 깃 이력에 기록되는 상황을 방지할 수 있습니다.

```
.gitignore

# Environments
.env
.venv
env/
venv/
ENV/
env.bak/
venv.bak/
```

5. 수정한 gpt_basic.py 파일을 실행해 보면 문제없이 결과가 출력됩니다.

```
ChatCompletion(id='chatcmpl-AJAiIKtkc96V0wj2qBwpl4abIimtl', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='2022년 FIFA 월드컵에서는 아르헨티나가 우승을 차지했습니다. 아르헨티나는 결승전에서 프랑스를 상대로 승리하여 월드컵 트로피를 들어올렸습니다.', refusal=None, role='assistant', function_call=None, tool_calls=None))], created=1729132778, model='gpt-4o-2024-08-06', object='chat.completion', service_tier=None, system_fingerprint='fp_a20a4ee344', usage=CompletionUsage(completion_tokens=52, prompt_tokens=30, total_tokens=82, completion_tokens_details=CompletionTokensDetails(audio_tokens=None, reasoning_tokens=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=None, cached_tokens=0)))
----
```

줄 9, 열 1 공백: 2 UTF-8 CRLF {} Python 3.12.1 ('venv': venv)